

M11 — Compute Module 5

Interaccion completa con el resto del payload INTISAT

Manual de la coleccion CubeSat (M11 / 11)

Coleccion CubeSat para el proyecto INTISAT

12 de mayo de 2026

Prefacio

Este manual responde una pregunta concreta: *cuando reemplacemos el Raspberry Pi 5 de desarrollo por el Compute Module 5 montado en un carrier custom, exactamente como interactua con el resto del payload INTISAT?*

No es un manual de PCB design (eso esta en `M10_CM5_Carrier_INTISAT.pdf`). No es un manual de electronica basica (eso esta en `Manual_Electronica_Practica.pdf`). No es un curso de Linux (eso esta en `Raspberry_Pi_5_Profundo.pdf`). Este manual cubre las **interacciones entre el CM5 y todo lo que lo rodea**: alimentacion, comunicacion con el OBC, control del sensor de imagen, control del OLED, monitoreo de potencia y boot sequence. Lo justo que necesitas saber para entender el sistema completo en funcionamiento.

Audiencia

- Ingenieros del payload que vienen del prototipo Pi 5 y quieren migrar al CM5.
- Cualquier miembro del equipo que pregunte "como anda eso".
- Operadores que tienen que diagnosticar problemas remotos.
- Lideres tecnicos que deben validar el diseño.

Prerequisitos

- `Manual_Electronica_Practica.pdf` (capitulos 1–12).
- `Raspberry_Pi_5_Profundo.pdf` (capitulos 1–7).
- `M10_CM5_Carrier_INTISAT.pdf` (al menos el cap. 1).
- Familiaridad con Python intermedio y Linux command line basico.

Como leer

El manual son 12 capitulos mas 3 apendices. Total ~ 35 paginas. Lectura completa: 1 dia laboral. Lectura selectiva: capitulos sueltos segun necesidad.

Cajas de color:

- **Verde** (Intuicion): analogias.
- **Gris** (Calculo): formulas y derivaciones.
- **Azul** (Conexion con INTISAT): como aplica a tu codigo.
- **Rojo** (Cuidado): errores comunes.
- **Naranja** (Ejercicios): preguntas para verificar.

Índice general

Prefacio	i
1. Vision general del sandwich CM5 + carrier	1
1.1. Por que importa entender esto	1
1.2. La forma fisica del sandwich	1
1.3. Las 5 funciones del carrier	2
1.4. El stack PC-104 del INTISAT	2
2. Los 200 pines del mezzanine	3
2.1. El conector DF40 de Hirose	3
2.1.1. Donde comprarlos	3
2.2. Categorías de pines	3
2.3. Cuales uso y cuales no para el INTISAT	4
2.4. El pinout completo (extracto)	4
3. Bloque de potencia detallado	6
3.1. El camino completo de la energia	6
3.2. Etapa 1: Fusible PTC reseteable	6
3.3. Etapa 2: Diodo Schottky de proteccion polaridad inversa	7
3.4. Etapa 3: LDO 5V o μ Module	7
3.4.1. Opcion A: LDO	7
3.4.2. Opcion B: μ Module (recomendada)	7
3.5. Etapa 4: INA219 monitor de potencia	8
3.6. Etapa 5: Pin 5V_IN del CM5	8
3.7. Etapa 6: PMIC interna del CM5	8
3.8. Capacitores de decoupling	8
3.9. Power-on sequence	8
4. Bus I2C con el OBC del INTISAT	10
4.1. Concepto y nivel logico	10
4.2. TXS0108E: el level shifter elegido	10
4.2.1. Conexion en el carrier	10
4.2.2. Configuracion del enable	11
4.3. Direccion I2C del payload	11
4.4. El BSC: I2C slave en el Pi	11
4.4.1.Codigo del slave (cubesat/i2c_slave.py)	11
4.5. Pull-up resistors	12
4.6. Velocidad de operacion	12
4.7. Codigo del master en el OBC simulado	12

5. Interfaz CSI-2 al sensor OV5647	14
5.1. Que es CSI-2	14
5.2. Características eléctricas	14
5.3. El cable FFC	14
5.4. Routing en el carrier	15
5.5. Configuración en Raspberry Pi OS	15
5.6. Código de captura (Python)	15
6. SPI al OLED SSD1351	17
6.1. SPI: 4 líneas + 2 de control	17
6.2. Conector en el carrier	17
6.3. Configuración en config.txt	17
6.4. Código de control (Python)	18
6.5. Velocidad de SPI	18
6.6. Timing del Display	18
7. Monitor INA219	19
7.1. Que mide y por que importa	19
7.2. Especificaciones	19
7.3. Conexión al carrier	20
7.4. I2C bus separado	20
7.5. Código de lectura	20
7.6. Calibración	20
8. Boot sequence	22
8.1. De power-on a daemon corriendo	22
8.2. Flashear la eMMC por primera vez	22
8.3. Debug por UART	23
9. Flujo de datos durante un scan completo	24
9.1. Timeline detallado	24
9.2. Datos generados	24
9.3. Downlink	25
10. Diferencias prácticas Pi 5 retail vs CM5	26
10.1. Que cambia, que NO cambia	26
10.2. Lo único que cambia en software	26
11. Procedimiento de bring-up	27
11.1. Step 1: inspección visual (15 min)	27
11.2. Step 2: tests eléctricos sin CM5 (30 min)	27
11.3. Step 3: montar el CM5 (15 min)	27
11.4. Step 4: primer boot via UART (30 min)	28
11.5. Step 5: validación de buses (20 min)	28
11.6. Step 6: instalar el daemon y validar (30 min)	28
11.7. Step 7: test funcional end-to-end (60 min)	28
12. Troubleshooting común	30
12.1. El CM5 no bootea	30
12.1.1. Síntoma: LED rojo encendido continuo, sin LED verde	30
12.1.2. Síntoma: LED verde parpadea N veces	30
12.2. El I2C no responde	30
12.2.1. Síntoma: i2cdetect no muestra 0x42	30

12.2.2. Sintoma: el master ve 0x42 pero los comandos no responden	30
12.3. La camara no se detecta	30
12.3.1. Sintoma: libcamera-hello -list-cameras no la muestra	30
12.4. El OLED no muestra nada	31
12.5. El INA219 lee 0	31
12.6. El consumo es mas alto del esperado	31
 A. Apendice A — Pinout completo del mezzanine	 32
 B. Apendice B — BOM del carrier (estimado)	 33
 C. Apendice C — Checklist pre-integracion al PC-104	 34
 Bibliografia	 35

Capítulo 1

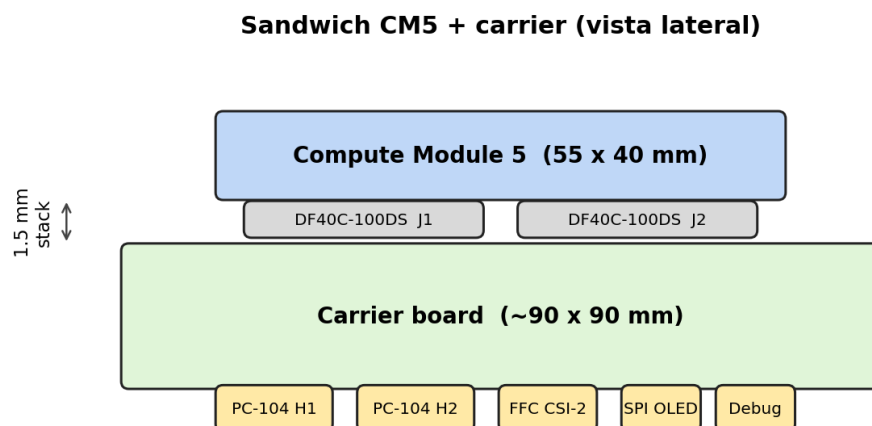
Vision general del sandwich CM5 + carrier

1.1. Por que importa entender esto

El Raspberry Pi 5 retail es un sistema *cerrado*: vos enchufas el USB-C y todo funciona. El **CM5 es un sistema abierto**: viene en formato modulo, sin alimentacion propia, sin conectores externos. Es **tu trabajo** diseñar la PCB que provea todo eso: el *carrier board*.

Entender la interaccion CM5 + carrier es entender como pasa de "computadora en una caja" a "subsistema integrado en un satellite".

1.2. La forma fisica del sandwich



El CM5 se enchufa hacia abajo en los dos conectores hembra DF40 montados en el carrier. Quedan a 1.5 mm de separacion (stack height nominal del DF40C-100DS-0.4V).

Intuicion

Pensa el CM5 como un *modulo SO-DIMM* de una notebook. Vos no diseñas la SO-DIMM, la enchufas en la motherboard. La motherboard es el carrier. Vos diseñas la motherboard.

1.3. Las 5 funciones del carrier

1. **Recibir alimentacion** del rail 5V_PAYLOAD del INTISAT (via conector PC-104).
2. **Regular esa alimentacion** y proteger al CM5 (LDO, fusible, diodo Schottky).
3. **Conectar el CM5 a los buses externos**: I2C al OBC, CSI-2 al sensor, SPI al OLED.
4. **Monitorear el consumo** con un sensor INA219.
5. **Proveer debug y boot**: UART exposed, boton de boot, LED de status.

1.4. El stack PC-104 del INTISAT

El INTISAT es un CubeSat 3U. Internamente tiene un stack de PCBs apiladas con el estandar PC-104:

Posicion en el stack	Funcion
Top	Payload (microscopia) — tu carrier con CM5
Middle 1	ADCS (control de actitud)
Middle 2	OBC central + COMMS
Bottom	EPS (paneles, bateria, distribucion)

Los 104 pines del PC-104 distribuyen todos los rails (5V, 3.3V, GND) y los buses (I2C_1, I2C_2, CAN, UART) entre todas las cards del stack.

Capítulo 2

Los 200 pines del mezzanine

2.1. El conector DF40 de Hirose

El CM5 expone toda su conectividad via 2 conectores hembra **Hirose DF40C-100DS-0.4V**:

Parametro	Valor
Pines	100 por conector $\times 2 = 200$ totales
Paso	0.4 mm
Stack height	1.5 mm (nominal)
Corriente max	0.5 A por pin
Voltaje max	30 V
Ciclos de mate	50 (no apto para conectores frecuentes)
Encapsulado	SMD, soldar con horno por reflow
Precio	USD 4 cada uno

2.1.1. Donde comprarlos

- **Mouser**: 855-DF40C-100DS-0.4V(81), USD 4.13.
- **Digikey**: H125185CT-ND, USD 4.07.
- **Arrow**: stock variable.

Cuidado

Si soldas el DF40 al revés (rotado 180°), el CM5 NO encaja. Marca la orientación en el silkscreen y verifica antes de soldar.

2.2. Categorías de pines

Los 200 pines del CM5 se dividen funcionalmente:

Categoria	Cantidad	Que llevan
Power IN	4	5V de entrada principal
Power OUT	4	3V3 y 1V8 (que el CM5 te da para perifericos)
GND	30	tierra, multiples para baja impedancia
GPIO general	28	los pines del header de la Pi 5 (BCM 0-27)
USB	16	4 puertos USB (2 USB 3.0 + 2 USB 2.0)
Ethernet	8	PHY signals (no incluye magnetics)
HDMI	19	video out (TMDS, I2C, hot plug)
DSI (display)	18	2 puertos para LCD/OLED grande
CSI (camera)	18	2 puertos camera (4 lanes c/u)
PCIe Gen 2	11	1 lane diferencial + clk + reset
SDIO	8	para SD externa (no usado en CM5 con eMMC)
RTC	2	VBAT + I2C del RTC
Control	6	nRUN (reset), nPOWER_BTN, EE-PROM_nDIS, LED, etc.
Reservado / NC	8	no conectar

2.3. Cuales uso y cuales no para el INTISAT

Funcion	Uso?	Pines	Justificacion
5V IN	Si	4	alimentacion principal del CM5
GND	Si	todos	tierra de retorno
3V3 OUT	Si	2-3	alimentar TXS0108E, INA219, OLED
I2C1 (GPIO 2/3)	Si	2	primario, hacia OBC
SPI0 (GPIO 8-11)	Si	4	control del OLED SSD1351
GPIO 18, 24	Si	2	DC y RST del OLED
CSI-2 (4 lanes)	Si	18	camara OV5647
USB	No	16	innecesario en orbita
HDMI	No	19	innecesario en orbita
Ethernet	No	8	innecesario en orbita
DSI	No	18	no necesitamos display
PCIe	No	11	no necesitamos SSD NVMe
SDIO	No	8	usamos eMMC interna
nRUN (reset)	Si	1	para reset hardware desde el carrier
RTC VBAT	Si	1	opcional, mantener hora con bateria de boton

Conexion con INTISAT

Lo importante: **de los 200 pines, solo usas ~ 50**. Los otros 150 quedan no-conectados en tu carrier. Esto te da flexibilidad para diseñar una PCB chica.

2.4. El pinout completo (extracto)

Los siguientes son los pines mas usados para tu carrier. Pinout completo: <https://datasheets.raspberrypi.com/cm5/cm5-datasheet.pdf>.

Pin J1	Funcion	Pin J2	Funcion
1	GND	1	GND
3,5	5V_IN	3,5	5V_IN
7,9	3V3_OUT	7,9	3V3_OUT
22	GPIO2 (I2C1 SDA)	—	—
24	GPIO3 (I2C1 SCL)	—	—
26	GPIO8 (SPI0 CE0)	—	—
28	GPIO9 (SPI0 MISO)	—	—
30	GPIO10 (SPI0 MOSI)	—	—
32	GPIO11 (SPI0 SCLK)	—	—
50	GPIO18 (PWM/DC)	—	—
54	GPIO24	—	—
—	—	71-89	CSI-2 cam0 (4 lanes + clk)
99,100	nRUN, GLOBAL_EN	—	—

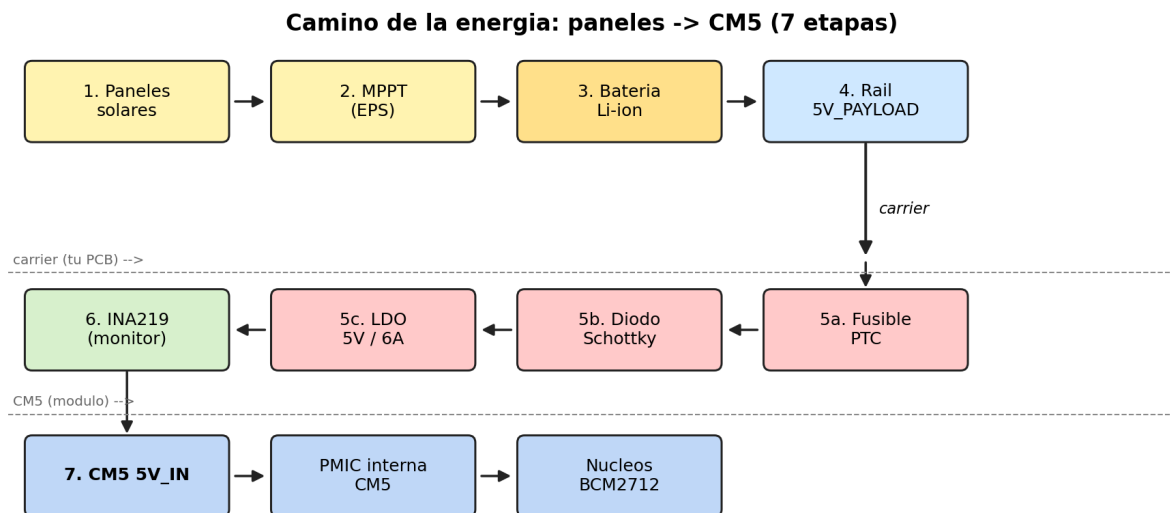
Importante: estos son los pines tipicos. La asignacion exacta varia con la version del firmware del CM5. *Verificar siempre contra el datasheet oficial.*

Capítulo 3

Bloque de potencia detallado

3.1. El camino completo de la energia

Desde los paneles solares del INTISAT hasta los nucleos del CM5, la energia atraviesa 7 etapas:



Tu carrier toma el rail 5V_PAYLOAD del PC-104 y, antes de pasar al CM5, le aplica 3 elementos de proteccion + 1 de medicion.

3.2. Etapa 1: Fusible PTC reseteable

- **Funcion:** si hay un corto en el CM5 o en algo aguas abajo, este fusible se "abre" (alta resistencia) y corta la corriente.
- **Tipo:** PTC (Positive Temperature Coefficient), reseteable. No se quema; se calienta y se enfria.
- **Eleccion:** MF-MSMF050-2 de Bourns, 5 A hold, 10 A trip. Cuesta USD 1.
- **Comportamiento:** a corriente nominal su resistencia es $\sim 0.02\ \Omega$ (negligible). En falla, $> 100\ \Omega$.

Calculo

Con 5 V y un corto que demanda 20 A, el PTC sube su resistencia a $0.25\ \Omega$:

$$V_{\text{PTC}} = I \cdot R = 20 \cdot 0.25 = 5\ \text{V}$$

Toda la tension cae sobre el PTC, la carga aguas abajo ve 0 V (apagada hasta que el PTC se enfrie).

3.3. Etapa 2: Diodo Schottky de proteccion polaridad inversa

- **Funcion:** si alguien conecta la fuente al revés, este diodo bloquea. Sin el, el CM5 se quema.
- **Tipo:** Schottky (bajo dropout, $\sim 0.3\ \text{V}$ vs $0.7\ \text{V}$ de Si normal).
- **Eleccion:** 1N5824 (3 A) o SS54 (5 A). Cuesta USD 0.50.
- **Caida:** $V_F \approx 0.3\ \text{V}$ a 1 A. Disipacion: 0.3 W.

3.4. Etapa 3: LDO 5V o μ Module

Despues del diodo entran $\sim 4.7\ \text{V}$ ($5\ \text{V} - 0.3\ \text{V}$ del diodo). El CM5 acepta $5.0\ \text{V} \pm 5\%$, asi que necesitamos volver a $5.0\ \text{V}$ con poca ripple.

3.4.1. Opcion A: LDO

- **Chip:** LT1086CT-5, 1.5 A, dropout $1.3\ \text{V}$ (no sirve a $4.7\ \text{V}$ de entrada).
- **Mejor:** TPS7A47, ultra-low-noise, dropout $\sim 0.5\ \text{V}$ (apenas alcanza).
- **Problema:** pierde la diferencia entre V_{in} y V_{out} como calor.

3.4.2. Opcion B: μ Module (recomendada)

- **Chip:** LTM4631 de Analog Devices.
- **Tipo:** buck-boost (sube si entra bajo, baja si entra alto).
- **Entrada:** $2.7\ \text{V}$ a $17\ \text{V}$.
- **Salida:** $5\ \text{V}$ @ $6\ \text{A}$.
- **Eficiencia:** 94% tipica.
- **Tamaño:** $9 \times 9 \times 4.92\ \text{mm}$ BGA.
- **Precio:** USD 25.

Calculo

Con LTM4631 a 94% y CM5 consumiendo $4\ \text{W}$:

$$P_{\text{disipada}} = P_{\text{out}} \cdot \frac{1 - \eta}{\eta} = 4 \cdot \frac{0.06}{0.94} = 0.26\ \text{W}$$

Con LDO equivalente, disiparia $(4.7 - 5.0) \cdot 0.8\ \text{A}$ pero como no puede subir voltaje, falla. Por eso el μ Module gana.

3.5. Etapa 4: INA219 monitor de potencia

Ver [Capítulo 7](#) para detalles. Resumen: mide voltaje, corriente y potencia del rail 5V que entra al CM5, comunicada por I2C secundario.

3.6. Etapa 5: Pin 5V__IN del CM5

Los 4 pines de 5V__IN del CM5 (J1 pin 3, 5 y J2 pin 3, 5) tienen que recibir la alimentacion regulada. Capacitor de bulk de 10 μ F y decoupling de 100 nF en cada uno.

3.7. Etapa 6: PMIC interna del CM5

El CM5 tiene una PMIC (Power Management IC) que genera todos los rails internos:

Rail	Voltaje	Para que
V_{CORE}	0.85–0.9 V	nucleos del CPU (DVFS variable)
V_{GPU}	0.9 V	GPU VideoCore VII
V_{DDR}	1.1 V	RAM LPDDR4X
V_{IO_18}	1.8 V	buses internos del SoC
V_{IO_33}	3.3 V	GPIO + perifericos externos

Estos rails internos NO son accesibles desde el carrier (excepto el 3V3 que se expone via mezzanine pin 7, 9). Son uso exclusivo del CM5.

3.8. Capacitores de decoupling

Regla universal: cada chip con voltaje propio necesita un capacitor de 100 nF lo mas cerca posible.

En tu carrier:

- 100 nF en cada pin 5V__IN del CM5 (4 unidades).
- 100 nF en cada pin 3V3 que use perifericos (TXS, INA219, OLED).
- Bulk 10 μ F al inicio del rail 5V (despues del LTM4631).
- Bulk 10 μ F al inicio del rail 3V3 (lo que provee el CM5).

Cuidado

Sin decoupling, los chips fallan intermitentemente bajo carga rapida. Es la regla numero uno de PCB design y se viola facilmente cuando uno empieza. Verificar en el layout antes de mandar a fabricar.

3.9. Power-on sequence

Cuando se aplica 5V al carrier:

1. $t = 0$: rail 5V estable en el carrier.
2. $t = 1$ ms: PMIC del CM5 genera V_{DDR} (1.1 V).
3. $t = 5$ ms: PMIC genera V_{CORE} y V_{IO} .

4. $t = 10$ ms: el SoC sale de reset.
5. $t = 100$ ms: el bootloader EEPROM empieza.
6. $t = 1$ s: bootloader carga `start.elf`.
7. $t = 5$ s: kernel Linux empieza a cargar.
8. $t = 30$ s: systemd termina de levantar servicios.
9. $t = 31$ s: tu daemon esta listo, I2C slave en 0x42.

Ejercicios

- 1) Calcular la potencia disipada por el LTM4631 cuando el CM5 esta en PROCESSING (7.5 W).
- 2) Si el rail 5V_PAYLOAD del INTISAT entrega solo 4.5 V (caida en cables), verifica que el LTM4631 sigue dando 5.0 V regulados.

Capítulo 4

Bus I2C con el OBC del INTISAT

4.1. Concepto y nivel logico

El OBC central del INTISAT y el CM5 hablan por I2C. Pero pueden estar operando a distintos voltajes:

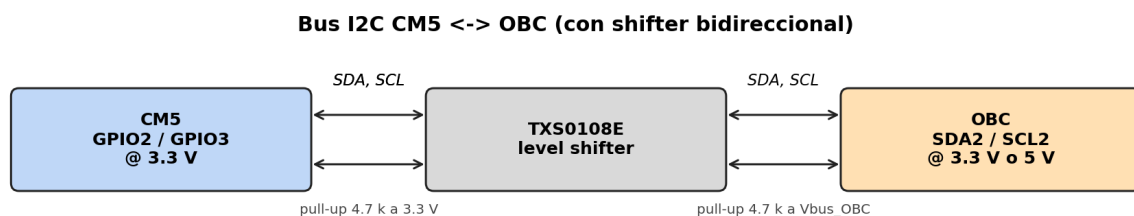
- OBC: típicamente 3.3 V o 5 V (depende del fabricante).
- CM5: 3.3 V en GPIO (no tolera 5 V directo).

Para conectarlos sin daño, se usa un **level shifter bidireccional**.

4.2. TXS0108E: el level shifter elegido

Parametro	Valor
Fabricante	Texas Instruments
Canales	8 (usamos 2: SDA y SCL)
Voltaje lado A	1.2 V a 3.6 V
Voltaje lado B	1.65 V a 5.5 V
Direccion	Auto (no necesita pin de control)
Velocidad max	110 Mbps
Encapsulado	TSSOP-20 o WQFN-20
Part number	TXS0108EPWR
Precio	USD 1

4.2.1. Conexion en el carrier



4.2.2. Configuracion del enable

El TXS0108E tiene un pin OE (Output Enable). Ata a 3V3 con un resistor de 10 k Ω y un capacitor de 1 μ F a GND para soft-start. Sin este RC, el shifter puede dar glitches al power-on.

4.3. Direccion I2C del payload

El I2C slave del CM5 responde en 0x42 (decimal 66). Esta direccion se eligio porque:

- NO esta reservada por el estandar I2C.
- NO la usa ningun chip estandar (sensores comunes usan 0x40-0x4F, 0x68-0x6F, etc.).
- Es facil de recordar.

Verificacion: ejecutar `i2cdetect -y 1` en el CM5 NO debe mostrar 0x42 (es slave, no master). Pero en el OBC, su `i2cdetect` si debe verlo.

4.4. El BSC: I2C slave en el Pi

El I2C estandar de Linux solo soporta master mode. Para hacer slave, el Pi 5 / CM5 tiene un periferico especial llamado **BSC** (Broadcom Serial Controller), accesible solo via `pigpio`.

4.4.1.Codigo del slave (cubesat/i2c_slave.py)

```

1 import pigpio
2 import time
3 from cubesat.commands import CMD_GET_STATUS, CMD_START_CAPTURE
4 from cubesat.commands import encode_status
5
6 I2C_ADDR = 0x42
7
8 def main():
9     pi = pigpio.pi()
10    if not pi.connected:
11        print("ERROR: pigpiod not running")
12        return
13
14    # Configurar BSC en modo slave
15    pi.bsc_i2c(I2C_ADDR)
16
17    while True:
18        # Lee bytes recibidos del master (OBC)
19        status, byte_count, data = pi.bsc_i2c(I2C_ADDR)
20
21        if byte_count > 0:
22            cmd = data[0]
23            payload = data[1:byte_count]
24
25            # Despachar segun comando
26            if cmd == CMD_GET_STATUS:
27                response = encode_status(...)
28                pi.bsc_i2c(I2C_ADDR, response)
29            elif cmd == CMD_START_CAPTURE:
30                # Encolar comando en /run/cubesat/commands/
31                pass
32            # ... mas comandos
33
34    time.sleep(0.01) # poll cada 10 ms

```



```

35
36 if __name__ == "__main__":
37     main()

```

4.5. Pull-up resistors

I2C requiere pull-up resistors en SDA y SCL porque ambos son **open-drain**: los chips solo pueden tirar la linea a 0 V; nadie puede ponerla a V_{cc} .

Valor tipico: 4.7 k Ω a V_{cc} (3.3 V en el lado A del TXS, V_{bus} en el lado B).

Calculo

Con 4.7 k Ω y 3.3 V:

$$I_{\text{pullup}} = \frac{3.3}{4700} = 0.7 \text{ mA}$$

A 100 kHz I2C, la capacitancia parasitica es ~ 50 pF. El tiempo de subida tipico es $R \cdot C = 235$ ns, ok para 100 kHz (periodo 10 μ s).

4.6. Velocidad de operacion

El I2C estandar opera a:

- **100 kHz** (Standard mode) — lo que usamos por defecto.
- **400 kHz** (Fast mode) — si el OBC lo soporta y los cables son cortos.
- **1 MHz** (Fast mode plus) — raro, requiere drivers especiales.

A 100 kHz: cada bit toma 10 μ s. Un byte (9 bits con ACK) toma 90 μ s. Un thumbnail de 100 KB toma ~ 10 segundos a transmitir.

4.7. Codigo del master en el OBC simulado

Para probar desde tu laptop o desde otra Pi:

```

1 import smbus2
2 import time
3
4 I2C_ADDR = 0x42
5 CMD_GET_STATUS = 0x01
6
7 bus = smbus2.SMBus(1)
8
9 # Enviar CMD_GET_STATUS
10 bus.write_byte(I2C_ADDR, CMD_GET_STATUS)
11 time.sleep(0.01)
12
13 # Leer respuesta (18 bytes: STATUS_OK + len + 16 bytes de status struct)
14 data = bus.read_i2c_block_data(I2C_ADDR, 0, 18)
15 print("Status:", data[0])
16 print("Length:", data[1])
17 print("Payload:", data[2:18])

```

Conexion con INTISAT

Este código es el que vas a usar para validar el bring-up del carrier antes de integrarlo al INTISAT. Lo corres desde otra Pi conectada a tu carrier via SDA/SCL y verifica que el daemon responde.

Capítulo 5

Interfaz CSI-2 al sensor OV5647

5.1. Que es CSI-2

CSI-2 = Camera Serial Interface 2. Es el estandar de MIPI Alliance para conectar camaras a procesadores. Lo usan todos los smartphones y todas las camaras de Raspberry Pi.

5.2. Caracteristicas electricas

- Hasta 4 lanes diferenciales de datos.
- 1 lane diferencial de clock.
- Voltaje: 200 mV pico a pico (D-PHY).
- Velocidad: 1.5 Gbps por lane.
- Impedancia: $90\ \Omega$ diferencial (con tolerancia $\pm 10\%$).

Para el OV5647 (2 lanes + 1 clock):

- DAT0+, DAT0-
- DAT1+, DAT1-
- CLK+, CLK-
- Mas: I2C control (CAM_SDA, CAM_SCL)
- Mas: poder (V_{ANA} , V_{IO} , V_{DIG})
- Mas: nRESET, PWDN, LED_CTRL

Total ~ 15 lineas.

5.3. El cable FFC

OV5647 se conecta al carrier por un cable plano flexible (FFC, Flat Flexible Cable):

- Estandar: 15-pin FFC, paso 1 mm.
- Longitud tipica: 15 cm.
- En el carrier: conector FFC vertical de 15 pines (Molex 5025810-15).

Cuidado

El conector FFC tiene polaridad. Los contactos pueden estar arriba o abajo. La OV5647 v1.3 tiene los contactos abajo. Tu carrier debe tener el conector con los contactos arriba para que el cable conecte correcto. Si te equivocas, no daña pero no funciona.

5.4. Routing en el carrier

Las líneas CSI-2 son criticas:

- Cada par (DAT0+, DAT0-) debe rutearse **adyacente** y de la misma longitud.
- Match de longitud entre los dos pares: ≤ 1 mm.
- Impedancia diferencial: $90\ \Omega$ (configurar en el stackup).
- Layer cercana al ground plane.
- Sin vias en el medio del par si es posible.

Distancia maxima del CM5 al conector FFC en el carrier: ≤ 100 mm (idealmente ≤ 50 mm).

5.5. Configuracion en Raspberry Pi OS

En /boot/firmware/config.txt:

```
1 # Habilitar camara
2 camera_auto_detect=1
3 dtoverlay=ov5647
4
5 # Opcional: I2C de la camara en bus distinto al primario
6 dtparam=i2c_vc=on
```

Despues del boot:

```
1 libcamera-hello --list-cameras
2 # Debe mostrar: 0 : ov5647 [...]
```

5.6. Codigo de captura (Python)

Usando Picamera2:

```
1 from picamera2 import Picamera2
2 import numpy as np
3
4 cam = Picamera2()
5
6 # Configuracion para microscopia (sin auto-correcciones)
7 config = cam.create_still_configuration(
8     main={"size": (2592, 1944), "format": "RGB888"},
9     raw={"size": (2592, 1944)}
10 )
11 cam.configure(config)
12
13 # Desactivar ISP para data cruda
14 cam.set_controls({
```

```
15     "AwbEnable": False,          # auto white balance OFF
16     "AeEnable": False,          # auto exposure OFF
17     "ExposureTime": 50000,      # 50 ms
18     "AnalogueGain": 1.0,        # sin amplificacion
19     "ColourGains": (1.0, 1.0),  # ganancia R/B = 1
20 })
21 cam.start()
22
23 # Capturar
24 img = cam.capture_array("main") # numpy array (H, W, 3)
25 print(f"Shape: {img.shape}, dtype: {img.dtype}")
```

Conexion con INTISAT

En tu daemon cubesat/capture.py, esto se llama 25 veces por scan FPM, una por cada angulo de iluminacion del OLED.

Capítulo 6

SPI al OLED SSD1351

6.1. SPI: 4 líneas + 2 de control

- **SCLK**: Serial Clock (GPIO11).
- **MOSI**: Master Out, Slave In (GPIO10). Datos del CM5 al OLED.
- **MISO**: Master In, Slave Out (GPIO9). *No usado* en OLED.
- **CE0**: Chip Enable (GPIO8). El OLED escucha solo cuando CE0 baja.
- **DC**: Data/Command (GPIO18). 0 = comando, 1 = data.
- **RST**: Reset (GPIO24). Activo bajo.

6.2. Conector en el carrier

Header de 9 pines en el carrier (paso 2.54 mm) hacia el OLED:

Pin	Señal	Tension
1	3V3	3.3 V
2	GND	0 V
3	MOSI	3.3 V logic
4	SCLK	3.3 V logic
5	CE0	3.3 V logic
6	DC	3.3 V logic
7	RST	3.3 V logic
8	V_OLED	3.3 V de alimentacion del panel (puede requerir 12 V boost interno)
9	GND	0 V

6.3. Configuracion en config.txt

```
1 # Habilitar SPI0
2 dtparam=spi=on
```

Despues:

```
1 ls /dev/spidev*
2 # /dev/spidev0.0
```

6.4. Código de control (Python)

Usando `luma.oled`:

```
1 from luma.core.interface.serial import spi
2 from luma.oled.device import ssd1351
3 from PIL import Image
4
5 # Setup SPI
6 serial = spi(port=0, device=0, gpio_DC=18, gpio_RST=24)
7 oled = ssd1351(serial, width=128, height=128)
8
9 # Crear imagen de iluminacion (un patron 5x5 con un pixel encendido)
10 img = Image.new("RGB", (128, 128), "black")
11 img.putpixel((64, 64), (255, 255, 255)) # pixel central encendido
12
13 # Mostrar
14 oled.display(img)
```

Conexion con INTISAT

En FPM, se enciende uno solo de los 25 puntos posibles del 5x5 antes de cada captura. Eso genera una "linterna" en diferente angulo. Tu código en `cubesat/capture.py` ejecuta este loop 25 veces.

6.5. Velocidad de SPI

El SPI0 puede operar de 0.5 MHz a 50 MHz. Para el OLED, 1 MHz es suficiente y reduce EMI.

6.6. Timing del Display

El SSD1351 tarda ~ 100 ms en actualizar la pantalla completa 128×128 . Para FPM, encender un solo pixel toma menos: ~ 10 ms.

Capítulo 7

Monitor INA219

7.1. Que mide y por que importa

El INA219 monitorea el rail 5V que alimenta al CM5. Provee:

- **Voltaje del bus:** cuanto voltaje recibe el CM5.
- **Voltaje del shunt:** caida sobre el shunt de $0.1\ \Omega$.
- **Corriente:** calculada como $V_{\text{shunt}}/R_{\text{shunt}}$.
- **Potencia:** $V_{\text{bus}} \cdot I$.

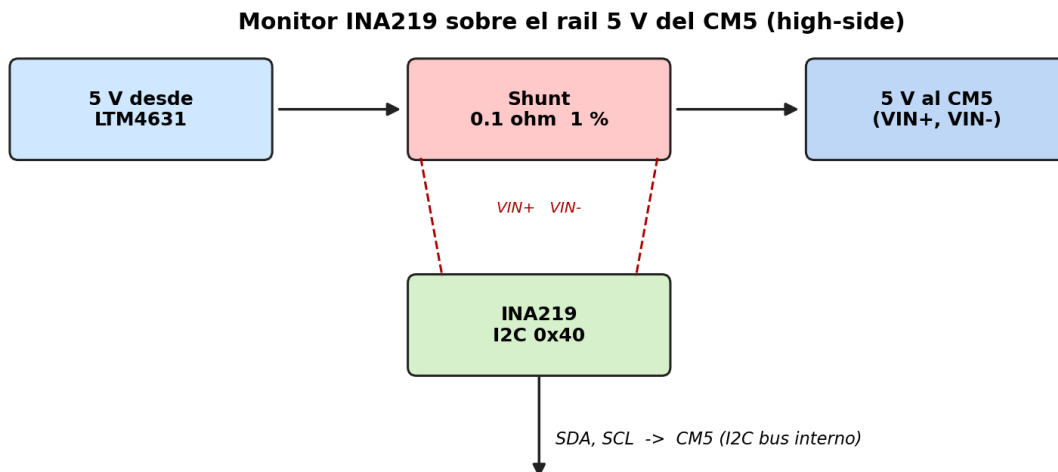
Aplicaciones:

- Telemetria: registrar consumo en cada modo (IDLE, CAPTURING, etc.).
- FDIR: detectar consumo anomalo (corto, regulador roto).
- Validacion: comparar consumo real vs estimado del datasheet.

7.2. Especificaciones

Parametro	Valor
Bus voltage max	26 V
Shunt voltage max	$\pm 320\text{ mV}$
Corriente max	$\pm 3.2\text{ A}$ (con shunt $0.1\ \Omega$)
Resolucion bus	4 mV
Resolucion shunt	$10\ \mu\text{V}$
Resolucion corriente	0.1 mA
Frecuencia conversion	hasta 1880 Hz
Comunicacion	I2C (4 address selectables)
Address default	$0x40$
Encapsulado	SOT-23-8 (chip) o modulo Adafruit
Precio (modulo)	USD 5

7.3. Conexion al carrier



7.4. I2C bus separado

El INA219 NO va en el mismo bus que el OBC. Razon: si el bus primario se cae (por colision con master remoto, glitch electrico), el daemon debe poder seguir leyendo telemetria.

Usar **I2C secundario** del CM5 (GPIO22/23 con dtoverlay i2c-gpio):

```
1 # en /boot/firmware/config.txt
2 dtoverlay=i2c-gpio,bus=2,i2c_gpio_sda=22,i2c_gpio_scl=23
```

7.5. Codigo de lectura

```
1 from adafruit_ina219 import INA219
2 import board
3 import busio
4
5 i2c = busio.I2C(board.SCL, board.SDA)
6 ina = INA219(i2c)
7
8 while True:
9     voltage = ina.bus_voltage + ina.shunt_voltage # V real
10    current = ina.current / 1000 # convertir mA a A
11    power = voltage * current # W
12
13    print(f"V={voltage:.3f}V I={current*1000:.1f}mA P={power:.3f}W")
14    time.sleep(0.5)
```

7.6. Calibracion

El INA219 tiene un registro de calibracion que define el factor de escala para la corriente. Por defecto viene calibrado para shunt $0.1\ \Omega$ y rango $\pm 3.2\text{ A}$. Si usas otro shunt, hay que reprogramar.

Conexion con INTISAT

Tu `tools/power_profiler.py` lee el INA219 cada 100 ms y guarda el log en CSV durante un scan. Es lo que vamos a usar para validar el presupuesto energetico real del payload.

Capítulo 8

Boot sequence

8.1. De power-on a daemon corriendo

Cuando el INTISAT enciende el rail 5V_PAYLOAD, el CM5 ejecuta esta secuencia:

1. **0–10 ms**: PMIC interna estabiliza rails.
2. **10–100 ms**: bootloader EEPROM (ROM interno) inicializa el SoC.
3. **100–500 ms**: bootloader carga el firmware del GPU desde la eMMC.
4. **500 ms–2 s**: GPU inicializa LPDDR4X, lee `config.txt` y `cmdline.txt`.
5. **2–5 s**: carga el kernel Linux desde la eMMC.
6. **5–20 s**: kernel descomprime, descubre hardware, monta rootfs.
7. **20–30 s**: systemd levanta servicios (pigpiod, networking, cubesat-*).
8. **30–40 s**: tu daemon esta listo, I2C slave responde.

Total: ~ 30 a 40 segundos.

8.2. Flashear la eMMC por primera vez

Para programar el sistema operativo en la eMMC del CM5:

1. Conectar el carrier por USB-C a una PC Linux.
2. Mantener pulsado el boton de nBOOT del carrier mientras se aplica alimentacion.
3. En la PC: `sudo rpiboot` (instalar previamente).
4. La eMMC aparece como `/dev/sda` en la PC.
5. Usar Raspberry Pi Imager para flashear Raspberry Pi OS Lite.
6. Configurar usuario, SSH, WiFi (este ultimo se desactivara despues).

8.3. Debug por UART

El CM5 expone una consola serial debug en GPIO14 (TX) y GPIO15 (RX). Para usar:

1. Conectar un adaptador USB-Serial (3.3 V) al header del carrier.
2. Abrir `screen /dev/ttyUSB0 115200`.
3. Ver los logs del boot en tiempo real.

Esto es **crucial** en bring-up porque permite ver mensajes de error antes que el sistema termine de arrancar.

Capítulo 9

Flujo de datos durante un scan completo

9.1. Timeline detallado

Asumiendo que el OBC del INTISAT envia `CMD_START_CAPTURE` (modo single):

Tiempo (ms)	Que pasa
0	OBC inicia transmision I2C: [START] [0x42] [0x02] [0x00]
2	TXS0108E shifter traduce voltajes 5V $\rightarrow$ 3V3
3	CM5 GPIO2/3 reciben los bits
5	pigpiod detecta byte recibido en BSC
10	cubesat-i2c-slave service lee el comando
12	escribe <code>cmd_20260507_101030.json</code> en <code>/run/cubesat/commands/</code>
15	responde al OBC: [STATUS_OK]
20	cubesat-pipeline service detecta nuevo archivo (inotify)
30	daemon cambia estado a CAPTURING
50	enciende OLED en posicion (64, 64) via SPI
200	OV5647 captura frame (50 ms exposicion + 150 ms processing)
250	CM5 escribe <code>cap_00.tiff</code> en <code>/var/cubesat/incoming/</code>
300	atomic rename <code>cap_00.tiff.tmp</code> $\rightarrow$ <code>cap_00.tiff</code>
350	daemon cambia estado a PROCESSING
400	carga modelo StarDist en memoria
1500	primera inferencia: segmenta el frame
2000	calcula metricas (areas, perimeters)
2500	escribe <code>summary.json</code> , <code>data.csv</code> , <code>overlay.png</code>
2800	daemon cambia estado a EXPORTING
2900	genera thumbnail 640x480 JPEG (<code>thumbnail.jpg</code>)
3000	atomic write <code>downlink/thumbnail.jpg</code>
3050	daemon vuelve a IDLE
3060	telemetry service publica el evento en <code>/run/cubesat/telemetry.json</code>

Total: $\sim$ 3 segundos en modo single. Comparado con $\sim$ 60 s en modo FPM.

9.2. Datos generados

Por cada scan single:

- 1 frame raw TIFF: $\sim$ 5 MB
- `overlay.png`: $\sim$ 500 KB

- mask.png: ~ 10 KB
- data.csv: ~ 5 KB
- summary.json: ~ 2 KB
- thumbnail.jpg: ~ 100 KB

Total disco: ~ 5.6 MB por scan single.

9.3. Downlink

Cuando el OBC pide `CMD_GET_THUMBNAIL`:

1. OBC: `write [0x42] [0x06]`
2. Daemon: lee thumbnail.jpg, lo trocea en chunks de 30 bytes.
3. Daemon: responde con frames `[OK] [30] [more=1] [chunk_0..29]`
4. OBC: lee 32 bytes por vez, acumula.
5. Total: ~ 3300 frames para 100 KB.
6. Tiempo total a 100 kHz: ~ 10 segundos.

Capítulo 10

Diferencias practicas Pi 5 retail vs CM5

10.1. Que cambia, que NO cambia

Aspecto	Pi 5 retail	CM5 + carrier
Sistema operativo	Raspberry Pi OS Lite Bookworm	idem
Kernel	6.6 + rpi	idem
Numeracion BCM	GPIO 2, 3, etc.	idem
Codigo del daemon	funciona	funciona (sin cambios)
Comandos shell	funcionan	funcionan
Conectores fisicos	USB-C + 4 USB-A + HDMI + Ethernet + header	via mezzanine al carrier
Almacenamiento	SD card	eMMC soldada
Como entra la energia	USB-C exterior	rail 5V del PC-104
WiFi/Ethernet	si, con conectores	desactivados en config
Boot	power-on con SD montada	power-on con eMMC
Flashear primera vez	SD a PC, escribir, insertar	USB-C a PC, rpiboot, flashear
Recovery	quitar SD, escribir nueva	rpiboot otra vez
Tamaño	85×56 mm	55×40 mm (mas tu carrier)
Peso	46 g	17 g + tu carrier

10.2. Lo unico que cambia en software

1. `config.txt`: agregar `dtoverlay=disable-wifi`, `dtoverlay=disable-bt`, configurar HDMI off.
2. `cubesat/install.sh`: detectar eMMC en lugar de SD.
3. Documentacion: actualizar guia de flasheo (rpiboot en lugar de Raspberry Pi Imager directo).

Capítulo 11

Procedimiento de bring-up

Cuando llega el primer carrier de fabrica:

11.1. Step 1: inspeccion visual (15 min)

- Microscopio o lupa 10x.
- Verificar todas las soldaduras: sin solder bridges, sin joints frios.
- Verificar componentes en sus footprints correctos.
- Verificar polaridad de capacitores polarizados (tantalio).
- Verificar orientacion de diodos (catodo marca).
- Verificar el silkscreen de los DF40 (matching del CM5).

11.2. Step 2: tests electricos sin CM5 (30 min)

Sin el CM5 montado, con una fuente externa:

1. Multmetro en modo continuidad: verificar GND continuo en todos los pines GND del carrier.
2. Multmetro en modo continuidad: verificar que NO hay corto entre 5V y GND, entre 3V3 y GND.
3. Alimentar con fuente 5 V, limite 100 mA. Verificar el rail 5V despues del LTM4631 = 5.00 ± 0.05 V.
4. Verificar que el INA219 responde: `sudo i2cdetect -y 2`, debe mostrar 0x40.
5. Subir el limite a 1 A, verificar que el rail no cae.

11.3. Step 3: montar el CM5 (15 min)

- Verificar orientacion: la flecha del CM5 alineada con la marca del carrier.
- Presionar uniformemente desde el centro hacia los bordes.
- Verificar stack height: $1.5 \text{ mm} \pm 0.2$.
- Atornillar con M3 si el carrier tiene agujeros de fijacion.

11.4. Step 4: primer boot via UART (30 min)

1. Conectar USB-Serial al header de debug del carrier.
2. Abrir `screen /dev/ttyUSB0 115200`.
3. Aplicar alimentacion 5 V.
4. Ver los logs en tiempo real.
5. Verificar que termina con prompt `login:`.

11.5. Step 5: validacion de buses (20 min)

Desde SSH al CM5 (despues de configurar):

```
1 # I2C primario (debe ver TXS0108E si tiene un sensor del lado externo)
2 sudo i2cdetect -y 1
3
4 # I2C secundario (debe ver INA219 en 0x40)
5 sudo i2cdetect -y 2
6
7 # SPI (lista de dispositivos)
8 ls /dev/spidev*
9
10 # Camara
11 libcamera-hello --list-cameras
12
13 # Lectura INA219
14 python -c "
15 from adafruit_ina219 import INA219
16 import board, busio
17 i2c = busio.I2C(board.SCL, board.SDA)
18 ina = INA219(i2c)
19 print(f'V={ina.bus_voltage:.2f} V, I={ina.current:.1f} mA')
20 "
```

11.6. Step 6: instalar el daemon y validar (30 min)

```
1 cd /opt
2 sudo git clone https://github.com/JaminYC/CubeSat-EdgeAI-Payload.git
3 cd CubeSat-EdgeAI-Payload
4 sudo bash cubesat/install.sh
5
6 # Verificar servicios
7 systemctl status cubesat-i2c-slave
8 systemctl status cubesat-pipeline
9 systemctl status cubesat-telemetry
10
11 # Probar comando I2C desde otra Pi
12 # (desde la otra Pi)
13 i2cset -y 1 0x42 0x01
14 i2cget -y 1 0x42 0
```

11.7. Step 7: test funcional end-to-end (60 min)

Desde otra Pi (simulando el OBC):

1. CMD_GET_STATUS: verificar respuesta valida.
2. CMD_START_CAPTURE (modo single): verificar que se crea archivo en `/var/cubesat/incoming/`.
3. Esperar el procesado.
4. CMD_GET_THUMBNAIL: descargar y verificar que es un JPEG valido.
5. Repetir 10 veces para validar repetibilidad.

Si los 7 steps pasan, el carrier esta listo para integrar al stack PC-104 del INTISAT.

Capítulo 12

Troubleshooting comun

12.1. El CM5 no bootea

12.1.1. Sintoma: LED rojo encendido continuo, sin LED verde

- **Causa probable:** alimentacion insuficiente.
- **Verificar:** voltaje en pin 5V_IN del CM5 con multmetro. Debe ser 5.00 ± 0.05 V.
- **Si el voltaje cae al cargar:** el LTM4631 no esta dando suficiente corriente. Verificar el inductor, los capacitores.

12.1.2. Sintoma: LED verde parpadea N veces

Codigos de error documentados en <https://github.com/raspberrypi/firmware/issues>.

- 4 parpadeos: no encuentra `start.elf` en la eMMC.
- 7 parpadeos: kernel image corrupta.
- 8 parpadeos: error de SDRAM.

12.2. El I2C no responde

12.2.1. Sintoma: `i2cdetect` no muestra 0x42

Recordar: el CM5 es slave, no aparece en su propio `i2cdetect`. Probar desde otra Pi (master).

12.2.2. Sintoma: el master ve 0x42 pero los comandos no responden

- Verificar que `pigpiod` esta corriendo: `systemctl status pigpiod`.
- Verificar que `cubesat-i2c-slave.service` esta running.
- Ver logs: `journalctl -u cubesat-i2c-slave -f`.

12.3. La camara no se detecta

12.3.1. Sintoma: `libcamera-hello -list-cameras` no la muestra

- Verificar conexion del cable FFC: contactos hacia arriba en el conector del carrier.
- Verificar `config.txt`: `dtoverlay=ov5647`.
- Verificar la I2C de la camara: `sudo i2cdetect -y 10` (bus camera), debe ver 0x36 (OV5647).

12.4. El OLED no muestra nada

- Verificar alimentacion 3V3 del OLED (multimetro).
- Verificar que el RST esta en alto (no en reset permanente).
- Probar con `luma.oled` ejemplo basico.

12.5. El INA219 lee 0

- Verificar shunt $0.1\ \Omega$ presente y sin corto.
- Verificar calibracion del INA219 (registro 0x05).
- Verificar polaridad: IN+ del lado de la fuente, IN- del lado del CM5.

12.6. El consumo es mas alto del esperado

- ¿WiFi desactivado? Verificar `rfkill list`.
- ¿BT desactivado? Verificar `hciconfig`.
- ¿HDMI off? Verificar `vcgencmd display_power 0`.
- ¿Throttling? Verificar `vcgencmd get_throttled`, no debe ser 0x0.

Apéndice A

Apendice A — Pinout completo del mezzanine

Pines mas usados del CM5 para el INTISAT:

Pin J	Numero	Funcion	Tipo	Notas
J1	1, 100	GND	power	ref electrica
J1	3, 5	5V_IN	power	alimentacion principal
J1	7, 9	3V3_OUT	power	3.3 V del CM5
J1	22	GPIO2 (SDA1)	I2C	SDA bus 1
J1	24	GPIO3 (SCL1)	I2C	SCL bus 1
J1	26	GPIO8	SPI	CE0
J1	28	GPIO9	SPI	MISO
J1	30	GPIO10	SPI	MOSI
J1	32	GPIO11	SPI	SCLK
J1	50	GPIO18	GPIO	DC OLED
J1	54	GPIO24	GPIO	RST OLED
J1	99	nRUN	ctrl	reset hardware (active low)
J1	100	GLOBAL_EN	ctrl	habilitacion global
J2	71	CAM_DAT0_P	CSI-2	lane 0 positivo
J2	72	CAM_DAT0_N	CSI-2	lane 0 negativo
J2	73	CAM_DAT1_P	CSI-2	lane 1 positivo
J2	74	CAM_DAT1_N	CSI-2	lane 1 negativo
J2	75	CAM_CLK_P	CSI-2	clock positivo
J2	76	CAM_CLK_N	CSI-2	clock negativo
J2	80	CAM_GPIO0	I2C	SDA camera
J2	82	CAM_GPIO1	I2C	SCL camera

Apéndice B

Apendice B — BOM del carrier (estimado)

Componente	Part number	Proveedor	USD
CM5 (4 GB/32 GB)	SC1230	RPi Trading	60
DF40 mezzanine (2x)	DF40C-100DS-0.4V	Hirose	8
μ Module 5V/6A	LTM4631	Analog Dev	25
Diodo Schottky 5A	SS54	Vishay	0.50
Fusible PTC 5A resetea- ble	MF-MSMF050	Bourns	1
INA219 modulo	INA219AIDR	TI	3
TXS0108E level shifter	TXS0108EPWR	TI	1
Cap. ceramico 100 nF (50x)	GRM188R71H104KA93D	Murata	2.50
Cap. tantalio 10 uF (10x)	T491A106K016AT	Kemet	3
Conector FFC 15-pin	5025810-09	Molex	1
Conector header 7-pin	estandar 2.54 mm	varios	0.50
OLED			
Boton SMD tactile	1825910-6	TE	0.50
LED bicolor SMD	APBA3025	Kingbright	0.50
Resistor 5.6 ohm 1 %	varios	varios	0.05
Resistor 4.7 k Ω 1 % (10x)	varios	varios	0.10
PCB 4-layer (10 unida- des)	JLCPCB	—	50

Total para 1 unidad: $\sim$ USD 156.

Apéndice C

Apendice C — Checklist pre-integracion al PC-104

- ☐ Inspeccion visual: sin solder bridges, todos los chips en footprint correcto.
- ☐ Continuidad: GND continuo, sin cortos a rails de alimentacion.
- ☐ Test sin CM5: el LTM4631 da 5.00 ± 0.05 V a vacio y con 1 A de carga.
- ☐ INA219 detectado en I2C bus 2 (0x40).
- ☐ CM5 montado correctamente con stack height 1.5 mm.
- ☐ Boot por UART completa hasta prompt de login.
- ☐ eMMC flasheada con Raspberry Pi OS Lite Bookworm.
- ☐ Configuracion aplicada: WiFi/BT/HDMI desactivados.
- ☐ Daemon `cubesat-pipeline` corriendo.
- ☐ I2C slave responde en 0x42 desde otra Pi (master).
- ☐ Camara OV5647 detectada por `libcamera-hello`.
- ☐ OLED SSD1351 responde a comandos via `luma.oled`.
- ☐ Test funcional: 10 scans en modo single sin error.
- ☐ OTA: subir un commit, verificar update + rollback.
- ☐ Watchdog systemd: simular cuelgue, verificar reset.
- ☐ Burn-in: 168 h corriendo sin reboot.
- ☐ Termico: T del SoC < 70 °C bajo carga sostenida.
- ☐ Documentacion: bring-up log firmado y archivado.

Bibliografía

1. Raspberry Pi Foundation. *Compute Module 5 Datasheet*.
<https://datasheets.raspberrypi.com/cm5/cm5-datasheet.pdf>, 2024.
2. Raspberry Pi Foundation. *CM5 IO Board Datasheet*.
<https://datasheets.raspberrypi.com/cm5/cm5-ioboard-datasheet.pdf>, 2024.
3. Hirose Electric. *DF40 Series Catalog*.
<https://www.hirose.com/en/product/series/DF40>.
4. Analog Devices. *LTM4631 Datasheet*.
<https://www.analog.com/en/products/ltm4631.html>.
5. Texas Instruments. *TXS0108E Datasheet*.
<https://www.ti.com/product/TXS0108E>.
6. Texas Instruments. *INA219 Datasheet*.
<https://www.ti.com/product/INA219>.
7. OmniVision. *OV5647 Datasheet*, 2009.
8. Solomon Systech. *SSD1351 Datasheet*, 2014.
9. MIPI Alliance. *D-PHY Specification v2.5*, 2022.
10. Manual M0 — CubeSat 101.
11. Manual M2 — Sistemas de Potencia EPS.
12. Manual M5 — Flight Software.
13. Manual M10 — CM5 + Carrier (diseño del PCB).